



FACULTAD DE INGENIERÍA Y ARQUITECTURA
INGENIERÍA EN SISTEMAS
ALGORITMOS Y ESTRUCTURAS DE DATOS

Proyecto de Investigación Análisis Radix Sort y Búsqueda
Binaria

Genesis Nicole Ramos Zambrana

Allysson Gabriela Palma Alvarado

Andres Humberto Porras Romero

Docente: Silvia Gigdalia Ticay López

Managua, 09 de julio de 2025

Índice

Introducción	1
Planteamiento del problema	2
Objetivo de la investigación	2
Objetivos específicos	2
Metodología	3
Marco conceptual / referencial	5
Implementación del algoritmo	7
Análisis a Priori	8
Análisis a Posteriori	10
Resultados	11
Conclusiones	12
Anexos	12
Referencias bibliográficas	13

Introducción

La presente investigación tiene como propósito analizar y comparar la eficiencia de diversos algoritmos de ordenamiento y búsqueda fundamentales en la informática. Se busca evaluar su comportamiento al ejecutarse en distintos dispositivos, ya que el rendimiento puede variar según las capacidades del hardware.

A través de la implementación de un sistema de impresión de notas en el lenguaje de programación Python, se ejecutarán diferentes algoritmos de ordenamiento (como Radix Sort, Quick Sort) y de búsqueda (como Búsqueda Binaria, Búsqueda Lineal) en equipos con distintas especificaciones para medir su eficiencia temporal y espacial.

Este proyecto permite aplicar conocimientos teóricos en un contexto práctico, entendiendo las ventajas, limitaciones y comportamiento real de los algoritmos según el entorno en que se ejecutan.

Planteamiento del problema

La eficiencia de los algoritmos de ordenamiento y búsqueda es fundamental para aplicaciones que manejan grandes volúmenes de datos. Sin embargo, su rendimiento no solo depende del tipo de datos, sino también del dispositivo en el que se ejecutan.

Algunos algoritmos pueden ser más rápidos pero consumir más memoria, y este consumo puede impactar más en dispositivos con recursos limitados. Por ello, se propone analizar y comparar varios algoritmos en diferentes entornos para elegir los más eficientes según el caso.

Objetivo de la investigación

Analizar y comparar la eficiencia de algoritmos de ordenamiento y búsqueda ejecutados en distintos dispositivos, evaluando su desempeño en un sistema de impresión de notas.

Objetivos específicos

- Evaluar la eficiencia de los algoritmos radix sort y búsqueda binaria en relación al tiempo de ejecución y consumo de memoria
- Implementar un sistema de impresión de notas en Python utilizando el algoritmo Radix Sort el cual ordene los datos de los estudiantes incluyendo (Cif, edad y calificación), Aplicando el algoritmo de búsqueda binaria dentro del sistema ordenado para localizar eficientemente la nota de un estudiante específico, evaluando su rapidez y precisión.

- Comparar el rendimiento de los algoritmos diferentes entornos de ejecución implementados en función del tiempo de ejecución y uso de memoria considerando el tipo de dispositivo

Metodología

1. Diseño de la investigación

El estudio es de tipo experimental, ya que se modificarán intencionalmente los algoritmos y los dispositivos para observar cómo afectan el rendimiento. En un entorno controlado, con estos cambios se mide el impacto de diferentes volúmenes y estructuras de registro en el rendimiento de los algoritmos especialmente en cuanto al tiempo que se tardan en ejecutarse y la cantidad de memoria que se usa.

2. Enfoque de la investigación

El enfoque de la investigación es cuantitativo dado a que se fundamenta en la recolección y análisis de datos numéricos que permiten establecer comparaciones objetivas entre los algoritmos implementados, se utilizan métricas precisas como el tiempo en milisegundos y el consumo de memoria en kilobytes para evaluar la eficiencia computacional de los algoritmos Radix sort y búsqueda binaria en escenario distintos de carga.

3. Alcance de la investigación

El alcance de estudio es descriptivo y explicativo, es descriptivo porque se detallan las características técnicas y funcionales de los algoritmos, es explicativo ya que se busca comprender y justificar el comportamiento de estos algoritmos frente a diferentes condiciones de entrada en diferentes dispositivos identificando así patrones y causas del rendimiento observado en los algoritmos estudiados.

4. Procedimiento

- Selección de algoritmos a estudiar.

Se eligen dos algoritmos en este caso Radix sort como técnica de ordenamiento no comparativa y búsqueda binaria como método eficiente de localización sobre las listas ordenadas.

- Codificación e implementación en un lenguaje de programación.

Se desarrolla un algoritmo en el lenguaje de programación python debido a sus sintaxis accesible y herramientas integradas para la medición de tiempos y uso de memoria.

- Ejecución con distintos dispositivos.

Ejecución del sistema en distintos dispositivos con diferentes características de hardware.

- Medición de eficiencia.

Se emplean herramientas de cronometraje como time o timeit y evaluación de uso de memoria como memory_profiler para registrar los recursos consumidos por cada algoritmo.

- Análisis comparativo.

Se analizan los datos obtenidos para comparar el rendimiento de los algoritmos en términos de tiempo de ejecución y consumo de memoria extrayendo conclusiones sobre su eficiencia en diferentes contextos de carga computacional

Marco conceptual / referencial

El marco conceptual es el conjunto de teorías, conceptos y antecedentes que sustentan la investigación. Permite ubicar el problema dentro de un contexto teórico y ayuda a definir las variables y las relaciones entre ellas.

Algoritmo:

Según *Ferrovial* (s.f.), un algoritmo es un conjunto de instrucciones sistemáticas y previamente definidas que se utilizan para realizar una tarea. Estas instrucciones están ordenadas y delimitadas para seguir pasos específicos con el fin de alcanzar un objetivo. Todo algoritmo tiene una entrada, conocida como *input*, y una salida, conocida como *output*; entre ambas se encuentran las instrucciones o pasos a seguir, los cuales deben estar ordenados y ser una serie finita de operaciones que permiten obtener una solución determinada a un problema.

Orden:

Según *Displayr* (s.f.), el orden es cualquier proceso que implica organizar los datos en una secuencia coherente para facilitar su comprensión, análisis o interpretación. Este método es comúnmente utilizado para visualizar datos de manera que se facilite la comprensión de cada uno de ellos.

Análisis a priori:

Según *BYJU'S* (s.f.), el análisis a priori es un examen teórico de un algoritmo que se realiza antes de su implementación o de la recopilación de datos. Su propósito es evaluar la viabilidad o el rendimiento potencial del algoritmo sin necesidad de ejecutarlo.

Análisis a posteriori:

Según *Prezi* (s.f.), el análisis a posteriori se basa en el conjunto de datos recogidos a lo largo de la experimentación. Este tipo de análisis se realiza después de la recolección de datos utilizando herramientas como cuestionarios, entrevistas, entre otros.

Comparativa de algoritmos:

Según *Medium* (s.f.), la comparativa de algoritmos consiste en evaluar y seleccionar el algoritmo más adecuado para un problema específico. En esta evaluación se consideran criterios como eficiencia, precisión, escalabilidad, uso de memoria y facilidad de implementación.

Sistema:

Según diversas fuentes académicas en ingeniería y ciencias sociales, un sistema es un conjunto de elementos que se relacionan entre sí y actúan de forma específica y coordinada. Cualquier segmento de la realidad puede considerarse un sistema, siempre que sea posible distinguir sus componentes interrelacionados del entorno externo.

Búsqueda binaria:

Según *DataCamp* (2024), la búsqueda binaria es un potente algoritmo diseñado para encontrar eficientemente un valor dentro de un conjunto de datos ordenado. A diferencia de la búsqueda lineal, este algoritmo reduce el intervalo de búsqueda a la mitad con cada paso, lo que lo convierte en un método significativamente más rápido.

Radix Sort:

Según *GeeksforGeeks* (s.f.), Radix Sort es un algoritmo de ordenamiento lineal que organiza los elementos procesando cada dígito de forma individual. Es especialmente eficiente para ordenar enteros o cadenas con claves de tamaño fijo, ya que agrupa los elementos por el valor de cada dígito, desde el menos significativo hasta el más significativo, logrando así el orden final.

Análisis de algoritmo:

Según *Runestone Academy* (s.f.), el análisis de algoritmos se centra en compararlos en función de la cantidad de recursos computacionales que utilizan. El objetivo es determinar cuál algoritmo es más eficiente en el uso de dichos recursos, como tiempo de ejecución o memoria utilizada.

Python:

Según *Alura Cursos* (s.f.), Python es un lenguaje de programación de propósito general, lo que implica que puede utilizarse para desarrollar una amplia variedad de aplicaciones y no está limitado a una problemática específica.

Implementación del algoritmo

En esta fase se presenta el código fuente de los algoritmos estudiados, con explicación de su funcionamiento y estructura. Puede incluir fragmentos de código, diagramas de flujo o pseudocódigo.

En esta parte del código se aprecia donde se importan los módulos con los demás elementos y luego se ven dos funciones. La primera es imprimir registros muestra en pantalla una lista de registros (que contienen datos como cif, edad y calificación) en un formato ordenado y colorido para que sea fácil de leer. La segunda, buscar por cif, recorre esa misma lista para encontrar un registro específico que coincida con el cif que le pasas, usando una búsqueda lineal. En resumen, una función para mostrar todos los datos claramente y otra para buscar un dato en particular.

```
Main.py X
Main.py > menu_principal
1 # principal.py
2
3 import base_de_datos
4 import ordenamiento_radix
5 import busqueda_binaria
6 import time #Importamos La biblioteca de tiempo
7 import tracemalloc # Importamos La biblioteca para rastrear La memoria
8
9 # Variable global para saber cómo está ordenada la lista actualmente
10 orden_actual = None
11
12 def imprimir_registros(registros):
13     """Imprime los registros de estudiantes de forma legible."""
14     if not registros:
15         print("No hay registros para mostrar.")
16         return
17     print("-" * 50)
18     print(f"{'CIF':<15} | {'Edad':<5} | {'Calificación':<12}")
19     print("-" * 50)
20     for registro in registros:
21         print(f"{registro['cif']:<15} | {registro['edad']:<5} | {registro['calificacion']:<12}")
22     print("-" * 50)
23
24 def buscar_por_cif(registros, cif):
25     """Realiza una búsqueda lineal simple por CIF."""
26     for registro in registros:
27         if registro['cif'].lower() == cif.lower():
28             return registro
29     return None
```

Este fragmento de código corresponde a la función `menu_principal()`, que es el menú interactivo del programa. Cuando se ejecuta, muestra varias opciones para gestionar los registros, tales como ver el estado actual, mostrar, ordenar por calificación, ordenar por edad, buscar por cif, restaurar la lista al orden original y salir del programa. El usuario debe ingresar un número para seleccionar la opción deseada y así realizar diferentes operaciones sobre la lista de registros. Es un menú que permite al usuario escoger qué acción realizar en cada momento en forma fácil y estructurada.

```

Main.py > buscar_por_cif
30
31 def menu_principal():
32     """Función principal que ejecuta el menú interactivo."""
33     global orden_actual
34
35     print("Cargando base de datos de estudiantes...")
36     lista_estudiantes = base_de_datos.obtener_registros_estudiantes()
37     print(f"¡Base de datos cargada con {len(lista_estudiantes)} registros!")
38     print("\n¡Bienvenido al Sistema de Gestión de Notas!")
39
40     lista_original = list(lista_estudiantes)
41
42     tracemalloc.start() # -> INICIAMOS EL RASTREO DE MEMORIA
43
44     while True:
45         print("\n--- MENÚ PRINCIPAL ---")
46         print(f"Estado actual: Lista {'ordenada por ' + orden_actual if orden_actual else 'desordenada'}")
47         print("1. Mostrar primeros 20 registros")
48         print("2. Ordenar registros por Calificación")
49         print("3. Ordenar registros por Edad")
50         print("4. Buscar por Calificación (Búsqueda Binaria)")
51         print("5. Buscar por CIF (Búsqueda Lineal)")
52         print("6. Restaurar lista original (desordenada)")
53         print("7. Salir")
54
55         opcion = input("Seleccione una opción: ")
56
57         if opcion == '1':
58             print("\nMostrando los primeros 20 registros...")
59             # ... (código para mostrar los primeros 20 registros) ...

```

La función `_ordenamiento_conteo_para_radix` organiza una lista de objetos numéricos según un dígito específico determinado por un exponente, utilizando una clave que extrae el valor relevante de cada elemento. Primero, cuenta la frecuencia de cada dígito en la posición indicada por el exponente, luego acumula estas frecuencias para determinar las posiciones finales de los elementos. A continuación, reconstruye una lista ordenada según dicho dígito, preservando el orden relativo entre elementos con el mismo valor. Finalmente,

actualiza la lista original con los nuevos valores ordenados, preparando así la lista para la siguiente iteración del algoritmo de ordenamiento.

```
# ordenamiento_radix.py

def _ordenamiento_conteo_para_radix(lista, exponente, clave):
    """Función auxiliar para Radix Sort (privada)."""
    n = len(lista)
    salida = [0] * n
    conteo = [0] * 10

    for i in range(n):
        indice = (clave(lista[i]) // exponente)
        conteo[indice % 10] += 1

    for i in range(1, 10):
        conteo[i] += conteo[i - 1]

    i = n - 1
    while i >= 0:
        indice = (clave(lista[i]) // exponente)
        salida[conteo[indice % 10] - 1] = lista[i]
        conteo[indice % 10] -= 1
        i -= 1

    for i in range(n):
        lista[i] = salida[i]

def ordenar(lista, clave):
    """
    Ordena una lista de objetos usando Radix Sort basándose en una clave numérica.
    """
```

```
if not lista:
    return lista

valor_maximo = max(lista, key=clave)
numero_maximo = clave(valor_maximo)

exponente = 1
while numero_maximo // exponente > 0:
    _ordenamiento_conteo_para_radix(lista, exponente, clave)
    exponente *= 10
return lista
```

En este módulo la función buscar, que realiza una búsqueda binaria en una lista ordenada para encontrar un objeto ('objetivo') basado en una clave ('clave'). La función evalúa el valor medio del rango actual y lo compara con el valor buscado, ajustando los límites superior e inferior de la búsqueda según corresponda, hasta encontrar el objeto o determinar que no existe. Si el valor medio coincide con el objetivo, devuelve ese elemento; si no,

continúa la búsqueda en la mitad correspondiente. La función retorna el objeto si lo encuentra o None si no logra localizarlo.

```
busqueda_binaria.py > ...
1  # busqueda_binaria.py
2
3  def buscar(lista, objetivo, clave):
4      """
5      Realiza una búsqueda binaria en una 'lista' ordenada para encontrar un 'objetivo'.
6      Devuelve el objeto completo si lo encuentra, de lo contrario devuelve None.
7      """
8      bajo = 0
9      alto = len(lista) - 1
10
11     while bajo <= alto:
12         medio = (alto + bajo) // 2
13         valor_medio = clave(lista[medio])
14
15         if valor_medio < objetivo:
16             bajo = medio + 1
17         elif valor_medio > objetivo:
18             alto = medio - 1
19         else:
20             return lista[medio]
21
22     return None
```

Análisis a Priori

A continuación se presenta el análisis a priori del caso de estudio el cual es la implementación de un sistema de impresión de notas en python utilizando como radix sort para el ordenamiento de búsqueda y método búsqueda binaria.

1. Eficiencia espacial

La eficiencia espacial se refiere a la cantidad de memoria adicional que requiere un algoritmo para ejecutarse más allá de los datos de entrada.

Radix Sort no es un algoritmo de ordenamiento in-place, lo que significa que requiere memoria adicional para completar su ejecución. En cada pasada, utiliza estructuras auxiliares como listas o arreglos de conteo para clasificar los elementos según el dígito actual que se está procesando. En una implementación típica en base decimal (base 10), esto implica el uso de estructuras de almacenamiento temporales asociadas a los diez posibles valores (del 0 al 9). Para una lista de n elementos, el algoritmo puede requerir hasta $O(n+k)$ de espacio adicional, donde k es el número de posibles valores por dígito.

Búsqueda binaria es un algoritmo que tiene una eficiencia muy alta por que puede implementarse de manera iterativa utilizando una cantidad constante de memoria. Por lo tanto, su complejidad espacial es $O(1)$

En términos de eficiencia espacial, el sistema es relativamente eficiente, aunque Radix Sort requiere memoria adicional para sus estructuras auxiliares. Este consumo está justificado si se considera su alta eficiencia temporal en comparación con algoritmos de ordenamiento comparativos.

2. Eficiencia temporal

En radix sort la eficiencia depende de la longitud máxima del número en dígitos y del número de elementos, el número total de operaciones es proporcional a $O(d*n)$, donde d es el número de dígitos de los valores a ordenar, en el caso de notas son enteros ya que van de 0 a 100, d es constante por lo que se puede considerar una eficiencia de $O(n)$.

En la búsqueda binaria dado a que esta se aplica sobre una lista previamente ordenada su eficiencia es $O(\log n)$ ya que cada iteración descarta la mitad del espacio de búsqueda.

3. Análisis de Orden

Radix sort si se considera que d es un dato constante ya sea por las notas donde n es el número de elementos y d son los dígitos del número $O(d*n)$, o por el cif es $O(n)$ ya que en este caso no crece con n por que se considera un dato fijo

En el caso de búsqueda binaria es $O(\log n)$ esto significa que incluso para una lista muy grande el número de comparación sigue siendo bajo es por eso que su notación en big o es de esta forma.

Esto significa que el sistema está bien optimizado para aplicaciones que requiere tanto ordenamiento como de búsqueda eficientes a la hora de tener registros de distintas magnitudes de cadenas numéricas

Análisis a Posteriori

A continuación se presenta el análisis a posteriori de nuestra investigación en base al código implementado en el lenguaje de python.

Contexto del caso: Este sistema de gestión de notas desarrollado en Python está diseñado para trabajar con una base de datos simulada de 5.000 estudiantes. Cada estudiante tiene un CIF único, una edad y una calificación. La aplicación permite ordenar registros por edad o por nota utilizando el algoritmo **Radix Sort**, y permite buscar estudiantes por calificación mediante **búsqueda binaria**, y por CIF con una búsqueda lineal. En entornos educativos con gran volumen de datos y necesidad de respuestas rápidas, la eficiencia de estos algoritmos incide directamente en el rendimiento y escalabilidad del sistema. A continuación, se analiza su comportamiento real en tres escenarios típicos: mejor caso, caso promedio y peor caso.

1. Análisis del mejor caso

En el mejor caso los datos están previamente organizados o requieren un procesamiento mínimo, sin embargo radix sort no mejora con el ordena inicial de los datos ya que no compara elementos directamente sino que organiza por dígitos desde la posición menos significativa hasta la más significativa, por lo tanto aun cuando las calificaciones o edades ya están ordenadas el algoritmo realizara las mismas pasadas procesando todos los elementos digito por digito, la complejidad de este caso es $O(n*d)$

donde:

n es la cantidad de registro

d es el número de dígitos de valor más largo.

En cuanto a **búsqueda binaria**, el mejor caso ocurre cuando la calificación buscada está en el centro de la lista ordenada. En este escenario, el valor se encuentra en la primera comparación, con una complejidad de: $O(1)$

Aplicación al caso : Con radix sort El sistema de gestión de notas mantiene un rendimiento constante incluso si los datos ya estaban ordenados. Esto permite reutilizar listas ordenadas sin pérdida de eficiencia, lo cual es ideal en procesos automatizados donde los datos pasan por distintas etapas (por ejemplo, ordenamiento previo para reportes)

Con búsqueda binaria . Si el sistema se adapta para ordenar claves complejas, como un CIF extendido (por ejemplo, 2024-9876 Z), el número de dígitos procesados aumenta. Aun así, Radix Sort seguirá siendo más eficiente que algoritmos comparativos como Insertion o Selection Sort, que se degradan hasta $O(n^2)$

2. Análisis del caso promedio

En condiciones normales con datos numéricos de longitud moderada y distribuidos aleatoriamente como es el caso de las calificaciones generadas entre 40 y 100, radix sort mantiene un comportamiento eficiente y constante, su rendimiento no se ve afectado por el orden si no por la estructura interna de los valores (longitud de dígitos) la complejidad promedio es $O(n*d)$.

En la práctica la búsqueda binaria sobre la lista ordenada por calificación permite encontrar cualquier valor con pocas comparaciones incluso sin conocer el estado de los datos, esto hace que el sistema sea confiable para consultas frecuentes por parte de docentes o personal administrativo el cual el nivel de complejidad es $O(\log n)$.

Aplicación al caso: En radix sort Dado que el sistema genera las notas aleatoriamente con una semilla fija, el comportamiento del algoritmo es estable y reproducible. Además, la implementación usa una función auxiliar basada en Counting Sort, lo que acelera el ordenamiento por dígito sin necesidad de comparaciones, aumentando la eficiencia general.

En búsqueda binaria: sobre la lista ordenada por calificación permite encontrar cualquier valor con pocas comparaciones, incluso sin conocer el estado de los datos. Esto hace que el sistema sea confiable para consultas frecuentes por parte de docentes o personal administrativo.

3. Análisis del peor caso

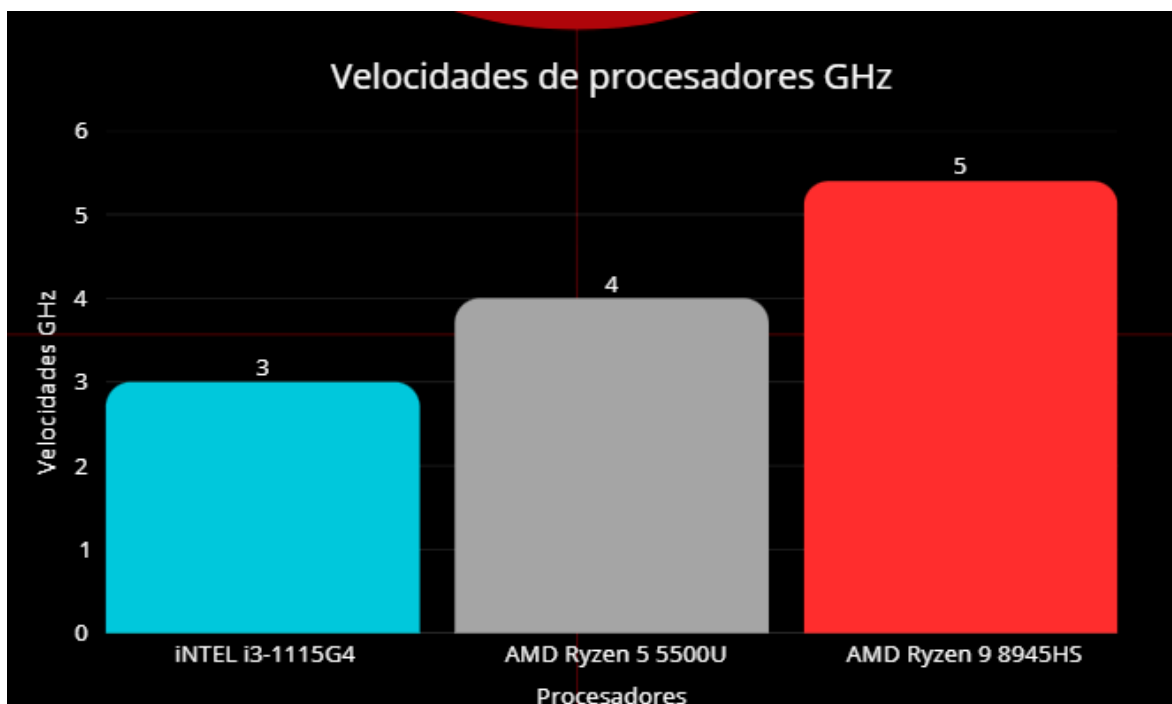
El peor escenario para radix sort no se debe al orden inverso, como sucede con algoritmos como bubble sort, sino a la longitud de los valores numéricos. Si el dato clave como el cif tiene muchos dígitos aumenta la cantidad de pasadas necesarias para completar la ordenación a pesar de eso la complejidad sigue siendo $O(n \cdot d)$ aunque con valores más elevados de d .

En el caso de la búsqueda binaria el peor caso se da cuando la calificación no existe en la lista o se encuentra en un extremo, en este caso el algoritmo debe dividir la lista logarítmicamente hasta descartar todas las posibilidades con una complejidad de $O(\log n)$.

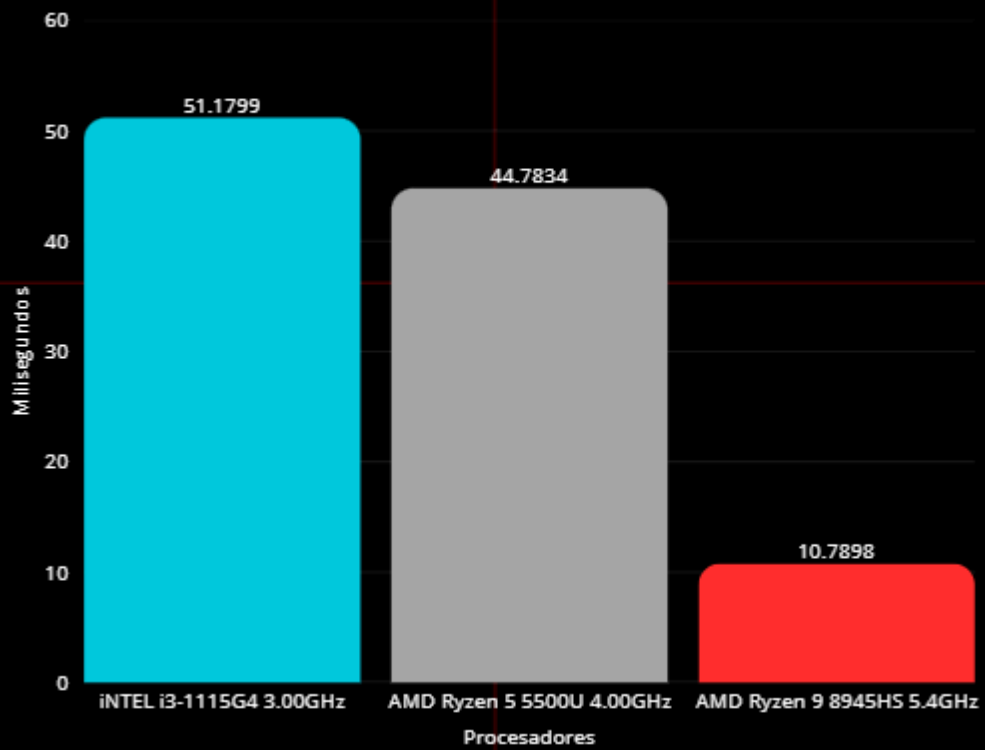
Aplicación al caso : con radix sort Si el sistema se adapta para ordenar claves complejas, como un CIF extendido (por ejemplo, 2024-9876Z), el número de dígitos procesados aumenta. Aun así, Radix Sort seguirá siendo más eficiente que algoritmos comparativos como Insertion o Selection Sort, que se degradan hasta $O(n^2)$.

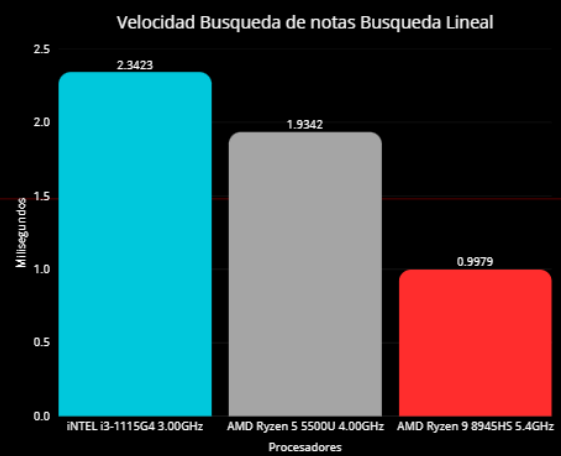
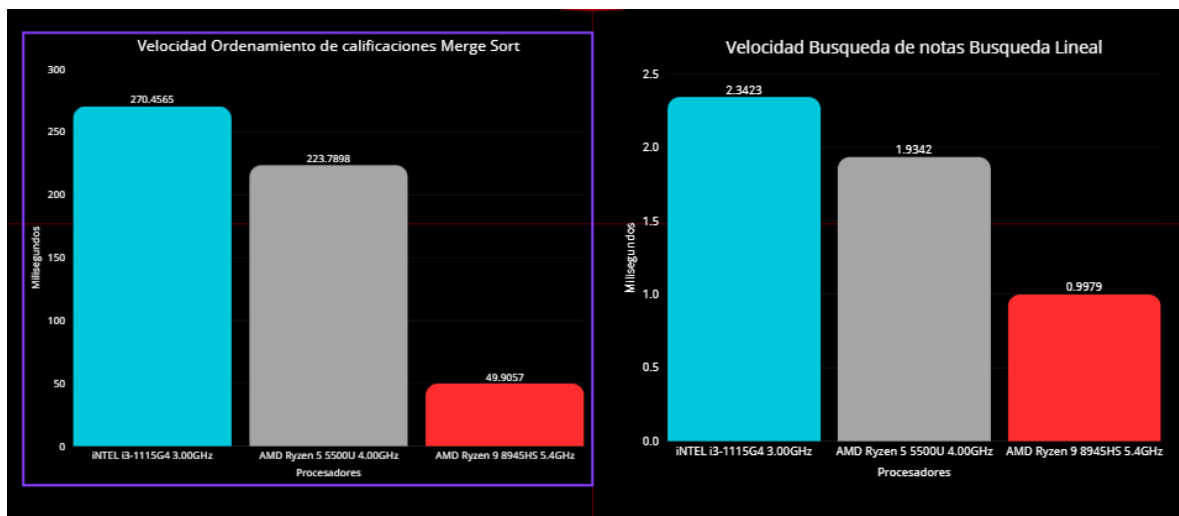
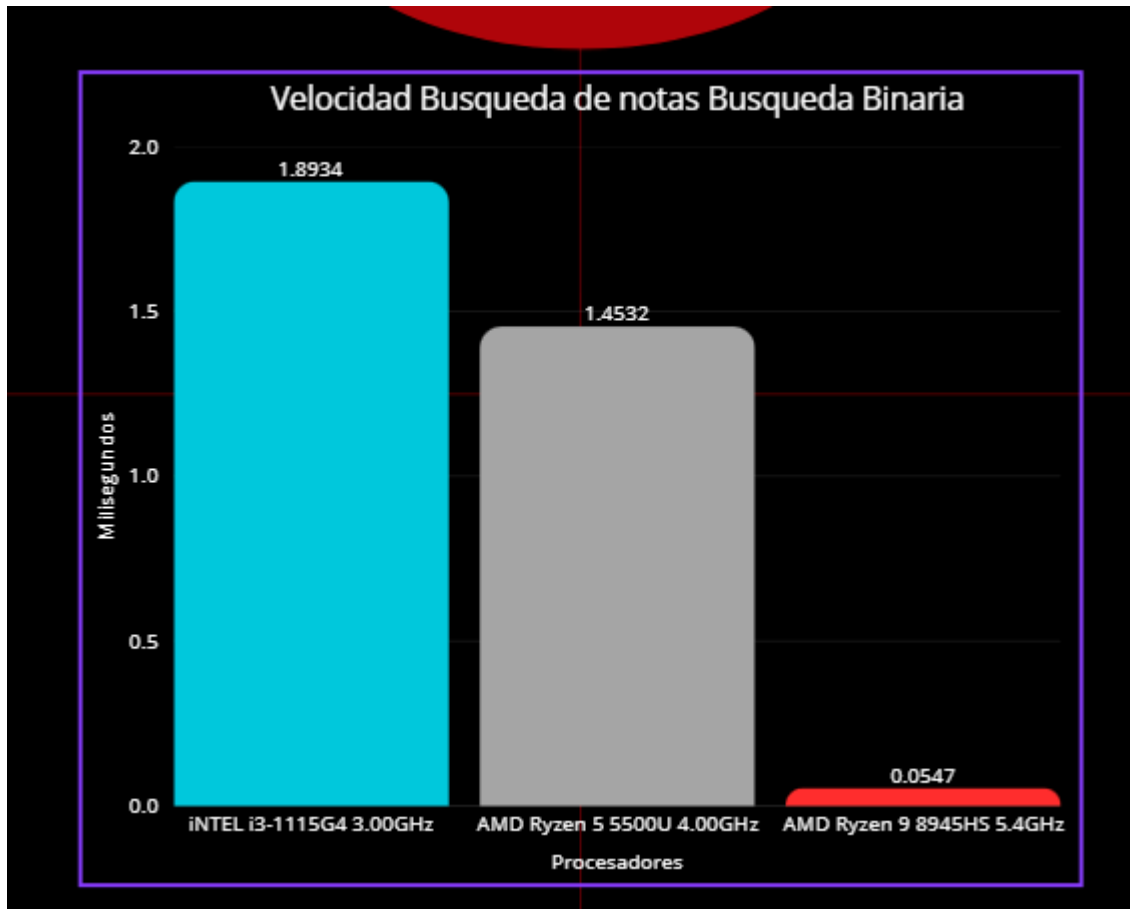
con Búsqueda binaria: Si un usuario busca una calificación atípica (por ejemplo, 39 o 101, que están fuera del rango generado), el sistema confirmará su ausencia tras realizar el número máximo de pasos posibles. Aun así, el tiempo de respuesta se mantiene en milisegundos, como se observa en las mediciones realizadas con `time.perf_counter()` y `tracemalloc`.

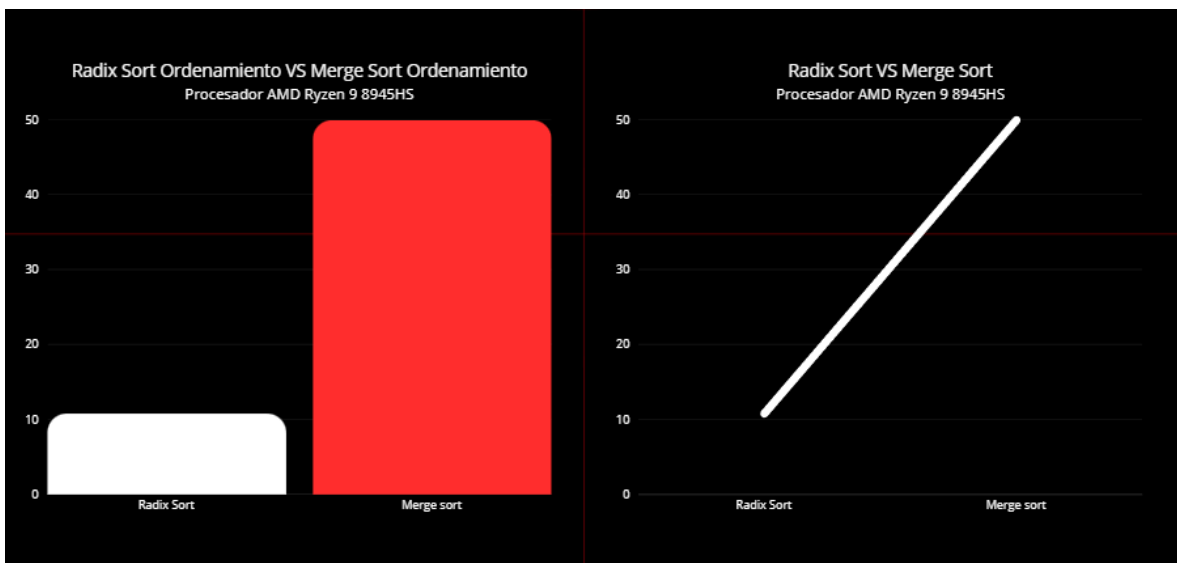
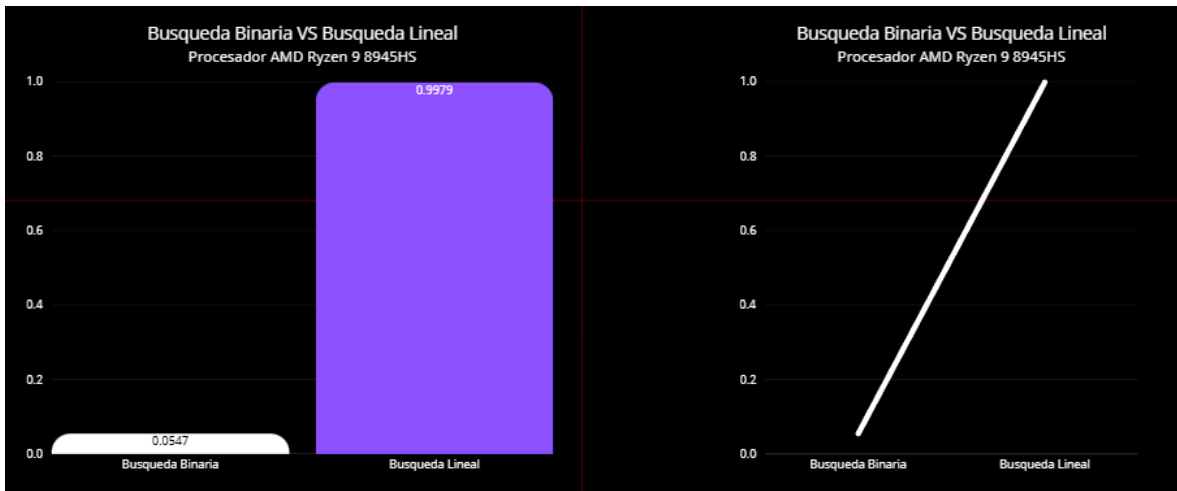
Resultados



Velocidad Ordenamiento de calificaciones Radix Sort







Metodo de ordenamiento por distintos algoritmos de búsqueda y ordenamiento

Marge sort vs Radix Sort

Ordenando con Merge Sort...



¡Lista ordenada con Merge Sort!

- Tiempo de ejecución: 49.9057 milisegundos.

- Pico de memoria usado: 79.0938 KB.

Ordenando con Radix Sort...



¡Lista ordenada con Radix Sort!

- Tiempo de ejecución: 10.7898 milisegundos.
- Pico de memoria usado: 39.7461 KB.

¿Por qué Radix Sort es más rápido aquí?

No hace comparaciones: Radix Sort no compara un número con otro. Simplemente distribuye los elementos en "cubetas" basándose en sus dígitos. Este proceso es mecánicamente muy rápido.

Complejidad Lineal para tus Datos: La complejidad de Radix Sort es $O(d \cdot n)$, donde d es el número de dígitos de tus números y n es la cantidad de datos. Tus calificaciones (máximo 100) y edades (máximo 25) tienen solo 2 o 3 dígitos. Esto hace que d sea un número muy pequeño y constante. Por lo tanto, el rendimiento se comporta de forma casi lineal, es decir, es directamente proporcional al número de registros.

Coste de Merge Sort: Merge Sort tiene una complejidad de $O(n \log n)$. Aunque es muy eficiente, el factor $\log n$ y el coste de dividir y fusionar las listas recursivamente introducen una sobrecarga que, para este tipo de datos, resulta ser mayor que el simple proceso de distribución de Radix Sort.

Metodo de busqueda

Búsqueda Binaria vs Búsqueda Lineal

Buscando con Búsqueda Binaria...

🕒 Búsqueda completada con Búsqueda Binaria.

- Tiempo de ejecución: 0.0547 milisegundos.

- Pico de memoria usado: 0.3125 KB.

🎉 ¡Estudiante encontrado!

CIF | Edad | Calificación

2022-9838F | 19 | 99

Buscando con Búsqueda Lineal...

🕒 Búsqueda completada con Búsqueda Lineal.

- Tiempo de ejecución: 0.9979 milisegundos.

- Pico de memoria usado: 0.2031 KB.

🎉 ¡Estudiante encontrado!

CIF | Edad | Calificación

2019-2179M | 21 | 99

¿Por qué la Búsqueda Binaria es tan superior?

La clave está en cómo cada algoritmo aborda el problema de encontrar un elemento.

1. Búsqueda Lineal: El método "Fuerza Bruta"

La Búsqueda Lineal es el método más simple. Funciona como buscar un libro en un estante desordenado: empiezas por el primero y revisas cada uno, uno por uno, hasta que encuentras el que buscas.

Proceso: Revisa el elemento 1, luego el 2, luego el 3, y así sucesivamente.

Complejidad: $O(n)$. En el peor de los casos (si el elemento está al final o no existe), tiene que recorrer toda la lista. Para tus 5,000 datos, esto significa que podría necesitar hasta 5,000 comparaciones.

2. Búsqueda Binaria: El método "Divide y Vencerás"

La Búsqueda Binaria es mucho más inteligente, pero necesita una condición fundamental: la lista debe estar ordenada. Tu sistema ya cumple esto al usar primero un método de ordenamiento.

Proceso:

Mira el elemento justo en el medio de la lista.

Si es el que buscas, ¡listo!

Si no, decide si tu objetivo está en la mitad izquierda o en la mitad derecha.

Descarta la mitad incorrecta y repite el proceso en la mitad restante.

Complejidad: $O(\log n)$. Cada paso reduce el problema a la mitad. Para tus 5,000 datos, el número máximo de comparaciones es $\log_2(5000)$, que es aproximadamente 13 comparaciones.

Conclusiones

En base a los análisis realizados y la implementación práctica del sistema realizado, evaluando así el comportamiento de ambos algoritmos aplicados en dos escenarios los cuales fueron ordenamiento por calificación y edad llegamos a la conclusión de que radix sort demostró ser un algoritmo eficiente para ordenar datos numéricos como calificaciones y edades, manteniendo un rendimiento constante en todos los casos gracias a su complejidad lineal $O(n*d)$.

La búsqueda binaria aplicada sobre la lista ordenada por calificación permitió localizar valores de forma rápida y con bajo consumo de recursos destacando por su eficiencia logarítmica de $O(\log n)$.

Aunque el sistema también emplea búsqueda lineal por cif, esta técnica es menos eficiente, sin embargo resulta útil en contextos donde no se requiere ordenamiento previo.

Las mediciones realizadas en tiempo de ejecución y uso de memoria muestran que ambos algoritmos se adaptan bien a bases de datos medianas como los 5000 registros utilizados en el sistema.

Anexos

<https://github.com/Ally1024/Trabajo-final-de-Algoritmo-y-estructura-de-datos/blob/Programa/Main.py>

Referencias bibliográficas

Ferrovial. (s.f.). *¿Qué es un algoritmo?* <https://www.ferrovial.com/es/stem/algoritmos/>

Displayr. (s.f.). *What is data sorting?*

<https://www.displayr.com/what-is-data-sorting/#:~:text=La%20ordenaci%C3%B3n%20de%20datos%20es...>

BYJU'S. (s.f.). *Algorithm analysis notes.*

<https://byjus.com/gate/algorithm-analysis-notes/#:~:text=Priori%20Analysis...>

Prezi. (s.f.). *Ingeniería didáctica como metodología de investigación.*

<https://prezi.com/a1l0vkw-tzx2/ingenieria-didactica-como-metodologia-de-investigacion/#:~:text=El%20an%C3%A1lisis%20a%20posteriori...>

DataCamp. (2024, septiembre 3). *Búsqueda binaria en Python: guía para una búsqueda eficiente.* <https://www.datacamp.com/es/tutorial/binary-search-python>

GeeksforGeeks. (s.f.). *Radix sort.* <https://www.geeksforgeeks.org/dsa/radix-sort/>

Alura. (s.f.). *¿Qué es Python? Historia y guía para iniciar.*

<https://www.aluracursos.com/blog/que-es-python-historia-guia-para-iniciar>

Runestone Academy. (s.f.). *¿Qué es análisis de algoritmos?*

<https://runestone.academy/ns/books/published/pythoned/AlgorithmAnalysis/QueEs>

[AnalisisDeAlgoritmos.html](#)

Data Overload. (s.f.). *Comparing algorithms: Choosing the right tool for the job.*

<https://medium.com/@data-overload/comparing-algorithms-choosing-the-right-tool-for-the-job-7a4d7d109265>

W3Schools. (s.f.).

Radix Sort Algorithm. https://www.w3schools.com/dsa/dsa_algo_radixsort.php

Runestone Academy. (s.f.).

La búsqueda binaria

<https://runestone.academy/ns/books/published/pythoned/SortSearch/LaBusquedaBinaria.html>

WsCube Tech. (s.f.).

Radix Sort in DSA: Explanation, Working, and Complexity.

<https://www.wscubetech.com/resources/dsa/radix-sort>

Khan Academy. (s.f.).

Búsqueda binaria.

<https://es.khanacademy.org/computing/computer-science/algorithms/binary-search/a>

[/binary-search](#)